

Internship Project Report
EDGE-BASED SMART SUSPENSION SYSTEM

ABSTRACT

Road anomalies like potholes and speed bumps degrade ride comfort, accelerate suspension wear, and compromise passenger safety. Traditional monitoring approaches are typically manual or reactive, identifying surface hazards only after vehicle-road interaction has occurred. To address these limitations, this project develops an edge-based computer vision framework for the real-time detection and proactive assessment of road anomalies using a single monocular camera.

The system implements a YOLOv8 object detection model, trained and validated on a custom annotated dataset, to isolate potholes and speed bumps from live video streams. Upon detection, a ground-plane projection model uses camera geometry to calculate the precise distance between the vehicle and the hazard. The system then estimates the physical width of the anomaly, evaluating its severity against predefined dimensional thresholds.

To enable predictive suspension control, the framework computes Time-to-Impact (TTI) by factoring in the estimated distance and current vehicle speed. A decision-making module integrates these severity and urgency metrics to generate proactive suspension adjustment directives before the vehicle reaches the disturbance. Localised processing on an edge device eliminates cloud dependencies, minimising communication latency and data overhead.

Experimental testing demonstrates reliable, real-time detection performance with minimal computational overhead. By leveraging a single monocular camera and localised edge computing, this framework offers a cost-effective alternative to the expensive predictive suspension systems found in premium vehicles. This work provides a scalable foundation for future integration with active suspension hardware, deep depth-estimation models, and intelligent autonomous vehicle platforms.

Table of Contents

CHAPTER 1: INTRODUCTION.....	5
1.1 Background.....	5
1.2 Problem Statement.....	5
1.3 Aim of the Project.....	5
1.4 Objectives.....	5
1.5 Proposed Methodology.....	5
1.6 Scope of the Project.....	5
1.7 Significance of the Project.....	6
1.8 Report Organisation.....	6
CHAPTER 2: LITERATURE REVIEW.....	7
2.1 Existing Systems.....	7
2.2 Research Papers Survey.....	7
2.3 Research Gap.....	7
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN.....	8
3.1 Proposed System.....	8
3.2 System Architecture.....	8
3.3 Data Flow.....	10
3.4 System Requirements.....	10
3.4.1 Hardware Requirements.....	10
3.4.2 Software Requirements.....	10
3.4 Set-Up.....	11
3.4.1 Environment Configuration.....	11
3.4.2 Dependency Installation.....	11
3.4.3 Installation Verification.....	12
3.5 Feasibility Analysis.....	12
3.5.1 Technical Feasibility.....	12
3.5.2 Economic Feasibility.....	12
3.5.3 Operational Feasibility.....	12
3.6 Chapter Summary.....	12
CHAPTER 4: METHODOLOGY.....	13
4.1 System Workflow.....	13
4.2 Dataset Preparation.....	15
4.3 Object Detection Module.....	15
4.4 Distance Estimation Module.....	16
4.5 Severity Assessment Module.....	17
4.6 Time-To-Impact Analysis.....	18
4.7 Decision Engine.....	19
4.8 Model Training Procedure.....	19
4.9 Edge-Based Deployment.....	20
CHAPTER 5: IMPLEMENTATION.....	21
5.1 Software Modules and Video Acquisition.....	21
5.2 Object Detection Module.....	21
5.3 Distance Estimation Module.....	21
5.4 Severity Assessment Module.....	22
5.5 Time-To-Impact Analysis.....	22
5.6 Decision Engine.....	22
5.7 Output Generation and Telemetry Serialisation.....	22
CHAPTER 6: RESULTS AND DISCUSSION.....	24
6.1 Experimental Setup.....	24
6.2 Dataset Statistics.....	24
6.3 Training Convergence.....	25
6.4 Class-Wise Performance Analysis.....	25
6.5 Detection Results.....	26
6.6 Distance Estimation Results.....	27
6.7 Severity Assessment Results.....	27
6.8 Time-to-Impact Analysis.....	27
6.9 Suspension Decision Results.....	28
6.10 Runtime Telemetry and Output Logs.....	28
6.11 Computational Throughput.....	28
6.12 Discussion.....	29
7. Conclusion and Future Scope.....	30

7.1 Conclusion.....	30
7.2 Limitations.....	30
7.3 Future Scope.....	30
Monocular Depth Estimation.....	31
Embedded Platform Deployment.....	31
Active Suspension Hardware Integration.....	31
CAN Bus and ECU Integration.....	31
Multi-Sensor Fusion.....	31
Adaptive Decision Logic.....	31
Digital Twin and Simulation Integration.....	31
Crowd-Sourced Road Mapping.....	31
References.....	32
Appendices.....	33
Appendix A — Dataset Samples.....	33
Appendix B — Model Training Configuration.....	34
Appendix C — Source Code Listings.....	35
<i>Listing C.1 — Ground-Plane Distance Estimation.....</i>	35
<i>Listing C.2 — Width Estimation and Severity Classification.....</i>	35
<i>Listing C.3 — Time-to-Impact Computation.....</i>	35
<i>Listing C.4 — Decision Engine.....</i>	35
Appendix D — Experimental Outputs.....	36
D.1 Detection Output Frames.....	36
.....	36
Detection Output — Pothole.....	36
D.2 CSV Log Sample.....	36
Final Report Structure.....	37

CHAPTER 1: INTRODUCTION

1.1 Background

Modern intelligent transportation systems has accelerated integration of artificial intelligence, computer vision, and edge computing to optimise vehicle dynamics. However, road surface irregularities like potholes and speed bumps continue to challenge ride quality. These surface anomalies introduce transient impact loads that accelerate suspension wear, compromise passenger comfort, and degrade vehicle stability.

Standard suspension systems operate reactively, responding only after striking a road hazard. While premium vehicles feature predictive suspension technologies, they typically depend on expensive sensors like LiDAR, radar, or stereo vision systems, which limits widespread adoption. This creates a distinct need for cost-effective, look-ahead solutions that detect hazards early and enable proactive suspension adjustment.

Advancements in deep learning-based object detection now allow for real-time road scene analysis, while edge-computing hardware provides the low-latency processing required for immediate vehicular responses.

1.2 Problem Statement

Drivers and automated systems frequently encounter road anomalies without warning, causing sudden suspension stress and mechanical wear. Current low-cost monitoring solutions focus primarily on basic hazard detection, omitting critical downstream tasks like spatial estimation, severity assessment, and decision support.

Without an integrated system that links real-time detection to actionable suspension directives, existing driver-assistance technologies remain reactive rather than proactive.

1.3 Aim of the Project

The objective of this project is to develop an edge-based vision framework that detects and assesses potholes and speed bumps in real time, generating predictive suspension adjustments before the vehicle encounters the anomaly.

1.4 Objectives

The key objectives of the proposed system are:

- Assemble and annotate a unified dataset containing pothole and speed bump images.
- Train and optimise a YOLOv8 object detection model for robust hazard identification.
- Implement real-time model inference on localised edge-computing hardware.
- Estimate hazard distance using monocular ground-plane projection.
- Assess anomaly severity using geometric features and spatial context.
- Calculate Time-to-Impact (TTI) based on vehicle velocity to prioritise risks.
- Generate proactive suspension adjustment directives using combined severity and urgency metrics.
- Log analytical data for performance verification and future optimisation.

1.5 Proposed Methodology

The framework uses a forward-facing monocular camera as its primary sensor. A custom-trained YOLOv8 model processes the video stream to identify road anomalies in real time. Once an object is detected, a geometric distance-estimation module applies ground-plane projection to calculate the distance between the vehicle and the anomaly. The system then estimates the physical width of the hazard to grade its severity and computes the Time-to-Impact (TTI) using the vehicle's current speed. Finally, a decision engine processes these parameters to generate proactive suspension adjustment commands.

1.6 Scope of the Project

This project covers real-time road hazard perception, distance estimation, severity assessment, and decision support executed entirely within an edge-computing environment. The scope is limited to detecting potholes and speed bumps, calculating their proximity, and generating control directives.

Physical suspension actuation, embedded automotive controller bus integration (e.g., CAN bus), and production-level hardware deployment are excluded from this phase and are reserved for future work.

1.7 Significance of the Project

The proposed system demonstrates the feasibility of integrating computer vision and edge intelligence for predictive vehicle dynamics applications. By eliminating dependence on high-cost sensing infrastructure, the framework provides a scalable and economically viable alternative for intelligent suspension assistance.

Furthermore, the project establishes a foundation for future research involving monocular depth estimation, active suspension control, autonomous driving integration, and advanced road condition analytics, thereby contributing to the evolution of next-generation intelligent transportation systems.

1.8 Report Organisation

This report is organised into eight chapters:

- Chapter 1 introduces the project background, problem statement, and objectives.
- Chapter 2 reviews relevant literature and existing road monitoring methodologies.
- Chapter 3 outlines the system analysis and design requirements.
- Chapter 4 details the architecture and framework design.
- Chapter 5 describes implementation details and model development.
- Chapter 6 presents experimental results and performance evaluations.
- Chapter 7 discusses future enhancements, and Chapter 8 concludes the report with key findings.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Systems

Road surface anomaly detection has gained significant attention due to its influence on vehicle performance, passenger comfort, and road safety. Traditionally, road condition assessment relied on manual inspections and scheduled maintenance surveys. Although effective for localised evaluation, these methods are labour-intensive, time-consuming, and unsuitable for continuous monitoring.

Subsequent approaches employed vehicle-mounted sensors such as accelerometers, gyroscopes, and vibration sensors to identify road irregularities through vehicle response analysis. While these systems reduced inspection effort, they remained inherently reactive, as anomalies were detected only after vehicle-road interaction.

In the automotive sector, premium manufacturers such as BMW, Mercedes-Benz, and Audi have introduced predictive suspension systems that utilise cameras, radar, and advanced control units to analyse road conditions ahead of the vehicle. These systems can adapt suspension characteristics before encountering a road disturbance; however, their reliance on specialised hardware significantly increases system complexity and cost.

Recent advances in computer vision and deep learning have enabled camera-based alternatives for road anomaly detection. Modern object detection architectures, including Faster R-CNN, SSD, and YOLO, provide accurate real-time detection of potholes and speed bumps using monocular vision systems. Furthermore, the adoption of edge computing has facilitated low-latency inference suitable for intelligent transportation applications.

Despite these developments, most existing solutions focus primarily on anomaly detection. Limited emphasis has been placed on integrating distance estimation, severity assessment, impact prediction, and suspension-oriented decision support within a unified edge-based framework.

2.2 Research Papers Survey

The research paper titled "Edge-Based Smart Suspension System Using Real-Time Road Anomaly Detection and Severity Assessment" proposes an intelligent suspension assistance framework capable of identifying potholes and speed bumps using computer vision techniques. The system employs a forward-facing camera and a deep learning-based detection model to recognise road anomalies in real time. Following detection, distance estimation and severity assessment are performed to evaluate the impact of the detected obstacle. The framework leverages edge computing to achieve low-latency processing and reduce dependence on cloud infrastructure. The study demonstrates the feasibility of utilising monocular vision for predictive suspension applications while maintaining a cost-effective system architecture.

2.3 Research Gap

Although existing predictive suspension and road anomaly detection systems have demonstrated promising results, many solutions rely on expensive sensing technologies such as LiDAR, radar, stereo vision systems, and proprietary automotive hardware. These requirements significantly increase deployment costs and limit their adoption to premium vehicle platforms.

Furthermore, several approaches depend on cloud-based processing architectures, introducing communication latency and reducing suitability for real-time vehicular applications. The need for continuous network connectivity also affects system reliability in practical operating environments.

Consequently, there exists a requirement for a cost-effective and self-contained solution capable of performing road anomaly detection and analysis directly on edge devices using a monocular camera. Such a framework can reduce hardware complexity, minimise operational costs, eliminate cloud dependency, and enable wider adoption of intelligent suspension assistance technologies across a broader range of vehicles.

To address these deficiencies, the proposed project introduces an Edge-Based Smart Suspension System that integrates real-time YOLOv8-based hazard detection, monocular distance estimation using ground-plane projection, severity assessment, Time-To-Impact computation, and suspension decision support. By combining these components within a single edge-computing framework, the proposed system extends beyond conventional detection-only methodologies and moves toward predictive vehicular intelligence.

3.1 Proposed System

The proposed system is an Edge-Based Smart Suspension Assistance Framework designed to detect and assess road surface anomalies in real time using computer vision and edge intelligence. The framework employs a forward-facing monocular camera to continuously acquire road scene imagery, which is processed locally through a deep learning-based perception pipeline.

A custom-trained YOLOv8 object detection model is utilised to identify potholes and speed bumps from incoming video frames. Following detection, geometric projection techniques are employed to estimate the longitudinal distance between the vehicle and the detected anomaly. The system further evaluates the physical dimensions of the hazard, classifies its severity level, and computes the corresponding Time-To-Impact (TTI) based on vehicle velocity.

Unlike conventional detection-only approaches, the proposed framework incorporates a decision-support module that transforms perception outputs into actionable suspension directives. This enables predictive assessment of road conditions prior to vehicle interaction, thereby establishing the foundation for future active and semi-active suspension systems.

Key Functional Components:

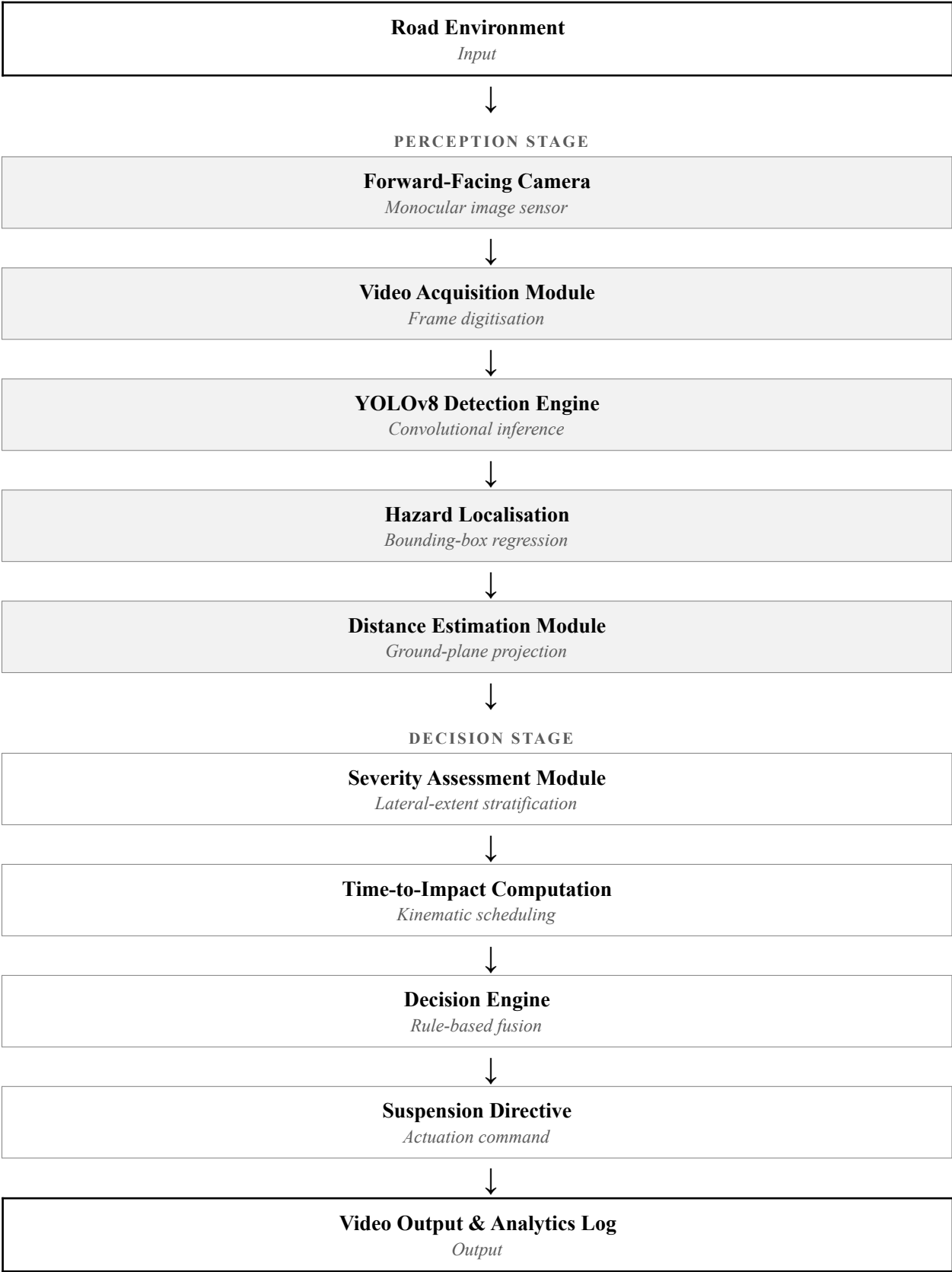
- Real-time road hazard detection.
- Monocular distance estimation.
- Hazard severity assessment.
- Time-To-Impact computation.
- Suspension decision generation.
- Edge-based processing and analytics logging.

3.2 System Architecture

The proposed framework is organised as a sequential, eleven-module processing pipeline that transforms a raw monocular video stream into a discrete suspension actuation directive. The architecture is partitioned into two functional stages, illustrated in Fig. 1: a perception stage, responsible for hazard detection and spatial localisation, and a decision stage, responsible for risk quantification and actuation scheduling.

A forward-facing monocular image sensor continuously observes the road segment ahead of the vehicle. Raw frames are digitised by the video acquisition module and supplied to the YOLOv8 detection engine, which performs single-pass convolutional inference to localise candidate road-surface anomalies — namely potholes and speed bumps — yielding bounding-box coordinates, class identity, and an associated confidence score for each instance. The resulting localisation is passed to the distance estimation module, where the lower edge of each bounding box is back-projected onto a planar ground-surface model to recover the longitudinal range separating the vehicle from the anomaly.

The severity assessment module subsequently derives the physical lateral extent of the detected feature from its pixel-domain dimensions and the estimated range, yielding a discrete severity classification. In parallel, the time-to-impact computation module combines the estimated range with the instantaneous vehicle velocity to determine the residual reaction interval available prior to wheel contact. Severity and urgency parameters are jointly evaluated by the decision engine, a rule-based fusion module that resolves a discrete suspension directive. The directive, together with the corresponding annotated frame, is persisted by the output and analytics logging module for subsequent visualisation and performance evaluation.



Flowchart. 3.2.1. Module-level architecture of the perception-to-actuation pipeline.

3.3 Data Flow

Table 1 enumerates the seven principal stages through which a single video frame is transformed into an actuation directive, together with the functional role discharged by each stage.

Table 3.3.1. Stage-wise description of the data-flow pipeline.

Stage	Designation	Functional Description
1	Data Acquisition	The image sensor continuously records the forward road scene, producing a sequential stream of digital frames for downstream processing.
2	Hazard Detection	Each frame is processed by the YOLOv8 detection network, which returns bounding-box coordinates, class identity, and a confidence score for every candidate anomaly.
3	Spatial Estimation	The lower edge of each bounding box is back-projected onto a planar ground-surface model to recover the longitudinal range to the detected anomaly.
4	Severity Analysis	Bounding-box dimensions, in conjunction with the estimated range, are used to approximate the physical lateral extent of the anomaly and assign a severity category.
5	Impact Assessment	Instantaneous vehicle velocity and estimated range are combined to compute the time-to-impact, denoting the residual reaction interval.
6	Decision Generation	Severity and urgency parameters are evaluated by a rule-based decision engine, which resolves a corresponding suspension directive.
7	Output Generation	Annotated frames and structured detection records are persisted for visualisation, auditing, and quantitative performance evaluation.

3.4 System Requirements

The implementation of the proposed framework presupposes computational resources capable of sustaining real-time deep-learning inference under low end-to-end latency, alongside a software environment tailored to vision-based deep learning development.

3.4.1 Hardware Requirements

Table 3.4.1.1. Hardware configuration of the perception and inference subsystem.

Component	Specification	Functional Role
Road Scene Acquisition Unit	Forward-facing monocular camera / recorded video source	Captures continuous road imagery ahead of the vehicle
Edge Communication Module	IP Webcam streaming interface	Provides low-latency wireless transmission of video frames
Edge Computing Node	CUDA - enabled processing workstation	Performs local inference without cloud dependency
Storage and Logging Module	SSD storage subsystem	Stores model weights, annotated videos, and decision logs
Visualisation Interface	Local display terminal	Presents detection, severity, and time-to-impact information

** Denotes a prospective requirement associated with the full edge-deployment configuration; the present implementation executes detection and decision logic on a single computational node.*

3.4.2 Software Requirements

The framework is developed using contemporary computer-vision and deep-learning libraries optimised for real-time inference, summarised in Table 3.

Table 3.4.2.1. Software environment and associated development tools.

Category	Tool / Library	Purpose
Operating System	Windows 11	Host environment for development and inference execution
Programming Language	Python 3.11	Primary implementation language, providing extensive support for machine learning and rapid prototyping
Deep Learning Framework	PyTorch (CUDA 12.8)	Underlying framework for model fine-tuning and GPU-accelerated inference
Detection Library	Ultralytics YOLOv8	Reference implementation employed for object detection and model deployment
Computer Vision Library	OpenCV	Video acquisition, frame processing, annotation rendering, and output encoding
Dataset Management	Roboflow	Dataset aggregation, annotation verification, augmentation, and YOLO-format export
Development Environment	Visual Studio Code, Jupyter Notebook, Git	Source-code editing, interactive experimentation, and version control
Analytics Tools	NumPy, Pandas, CSV logging	Post-processing and quantitative evaluation of system-generated telemetry

3.4 Set-Up

The development and execution environment was configured through interpreter installation, virtual-environment isolation, GPU-accelerated dependency installation, and verification of CUDA availability. The complete procedure, executed on a Windows 11 host, is summarised below; an equivalent shell-script procedure is provided for Linux-based hosts.

3.4.1 Environment Configuration

Python 3.11 was installed and isolated within a dedicated virtual environment to prevent dependency conflicts with the system interpreter.

```
py install 3.11
py -3.11 --version

py -3.11 -m venv suspension_env
.\suspension_env\Scripts\Activate
```

Listing 3.4.1.1. *Interpreter installation and virtual-environment activation (Windows PowerShell).*

3.4.2 Dependency Installation

Owing to the Blackwell architecture of the RTX 5050 Laptop GPU, a nightly CUDA 12.8 build of PyTorch was required in place of the standard stable release. Remaining dependencies were installed via pip.

```
python -m pip install --upgrade pip

pip install --pre torch torchvision torchaudio

pip install opencv-python==4.10.0.84
pip install ultralytics
pip install roboflow
pip install matplotlib Pillow
pip install pandas numpy scikit-learn tqdm
```

Listing 3.4.2.1 *GPU-enabled dependency installation (PyTorch, OpenCV, Ultralytics, Roboflow, and supporting libraries).*

3.4.3 Installation Verification

GPU availability and library integrity were verified prior to model training and inference, in accordance with the expected outputs indicated below.

```
nvidia-smi

python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
# Expected: True NVIDIA GeForce RTX 5050 Laptop GPU

python -c "from ultralytics import YOLO; print('YOLOv8 OK')"
# Expected: YOLOv8 OK
```

Listing 3.4.3.1 CUDA and library verification commands, with expected console output.

3.5 Feasibility Analysis

A feasibility assessment was conducted to evaluate the practicality of implementing the proposed framework in real-world vehicular environments.

3.5.1 Technical Feasibility

The proposed system is technically feasible due to the availability of mature deep learning frameworks, high-performance edge hardware, and optimised object detection architectures.

The custom-trained YOLOv8 model achieved satisfactory detection accuracy while maintaining real-time inference performance. GPU acceleration significantly reduced computational latency, enabling practical deployment within edge-computing environments.

Furthermore, the use of monocular vision eliminates the requirement for expensive sensing modalities such as LiDAR and radar, thereby simplifying system integration.

3.5.2 Economic Feasibility

The proposed framework demonstrates strong economic viability due to its reliance on low-cost sensing hardware and open-source software technologies.

The primary sensing component consists of a single monocular camera, significantly reducing implementation costs compared to advanced perception systems requiring multiple sensors.

Additionally, software components such as Python, OpenCV, PyTorch, YOLOv8, and Roboflow Community datasets minimise licensing and development expenses.

As a result, the proposed architecture offers a cost-effective pathway toward predictive suspension intelligence.

3.5.3 Operational Feasibility

From an operational perspective, the system is capable of functioning under real-time conditions with minimal user intervention.

The perception pipeline continuously processes incoming video streams and automatically generates hazard assessments and suspension directives. The modular architecture further supports future integration with active suspension controllers, embedded systems, and autonomous vehicle platforms.

The use of edge processing ensures reduced communication latency, enhanced reliability, and uninterrupted operation in environments with limited network connectivity.

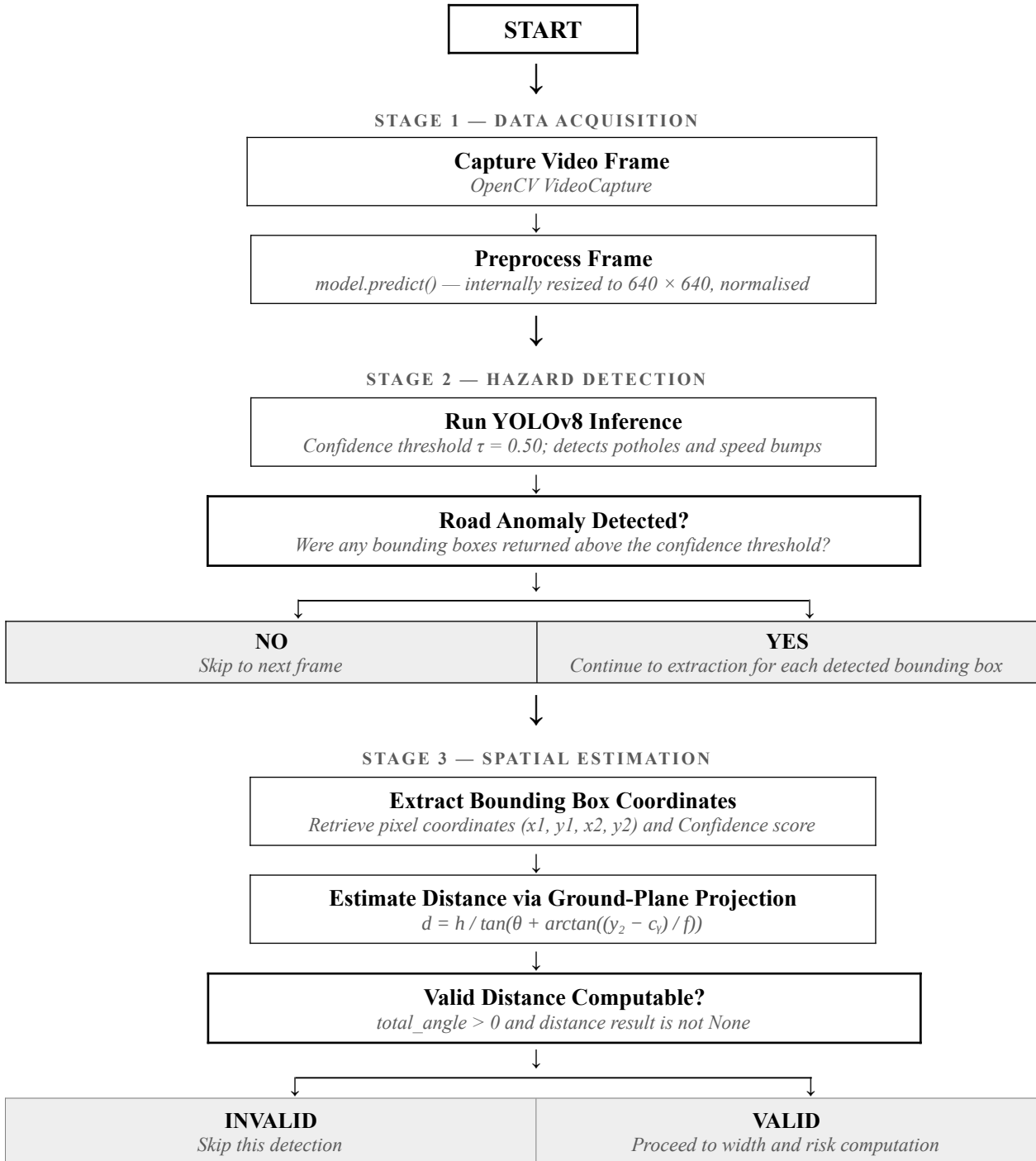
3.6 Chapter Summary

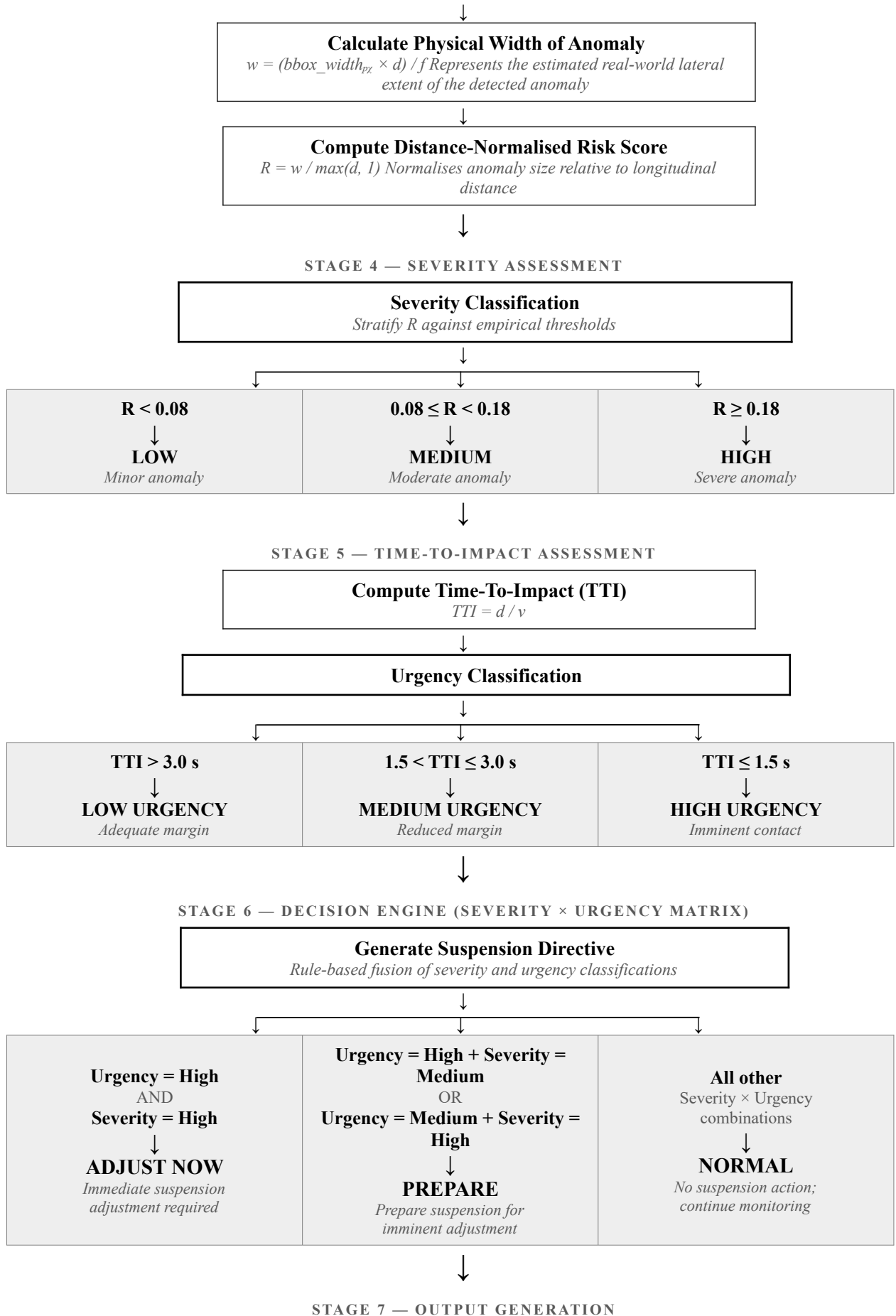
This chapter presented the analysis and design of the proposed Edge-Based Smart Suspension Assistance Framework. The overall architecture, data flow, hardware infrastructure, software environment, and feasibility considerations were discussed in detail. The analysis demonstrates that the proposed system is technically achievable, economically viable, and operationally suitable for real-time road hazard perception and predictive suspension decision support applications.

CHAPTER 4: METHODOLOGY

4.1 System Workflow

The proposed Edge-Based Smart Suspension System follows a sequential processing pipeline designed to identify roadway anomalies and generate suspension-oriented decision outputs in real time. Video frames acquired from a forward-facing monocular camera are processed locally on an edge computing platform. Each frame undergoes object detection, spatial analysis, severity assessment, and impact prediction before a corresponding suspension directive is generated. The complete workflow, including all conditional branches and decision outcomes, is presented in Fig. 4.1.1





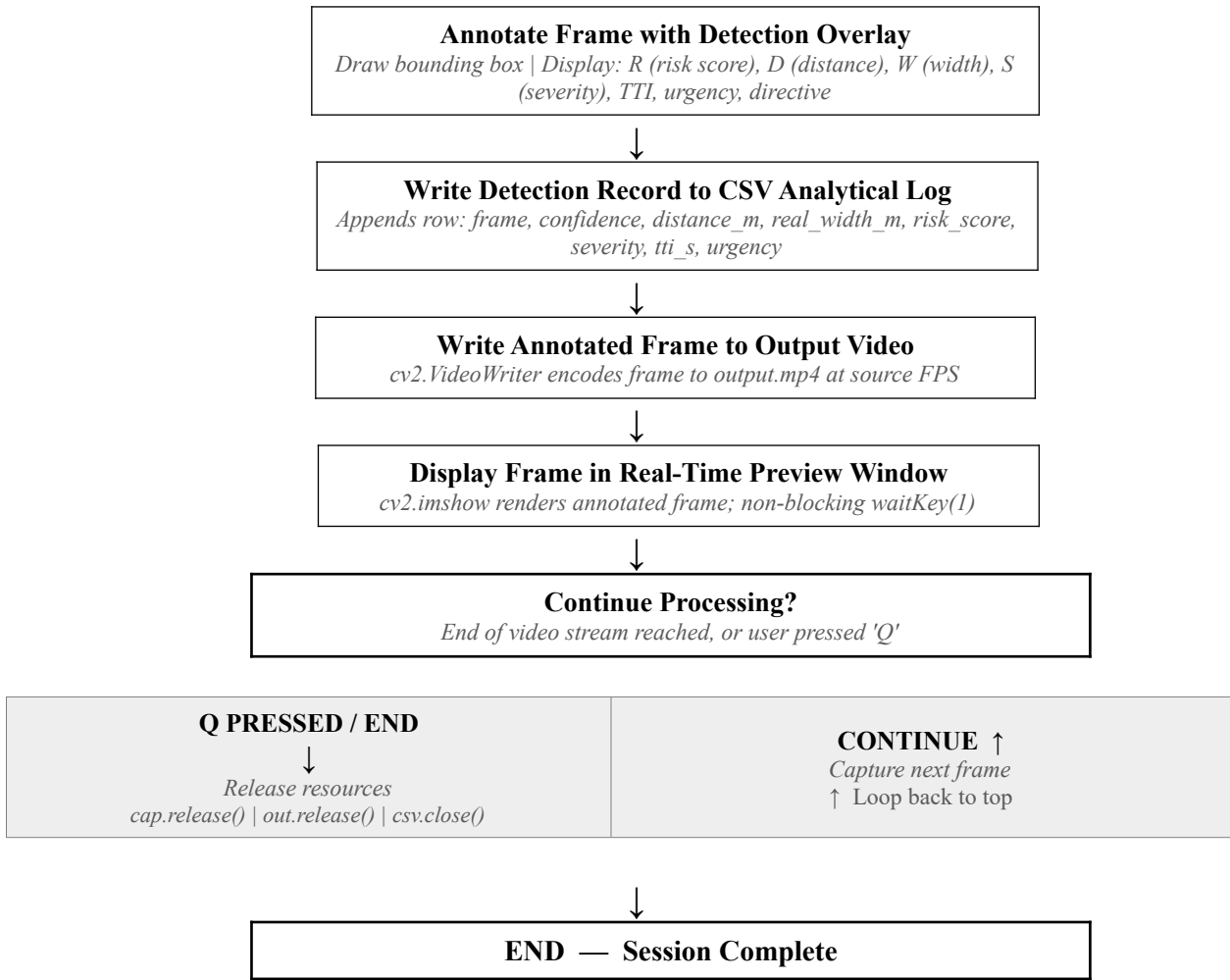


Fig. 4.1.1 Complete system workflow of the Edge-Based Smart Suspension System, including all conditional branches and suspension directive outcomes.

4.2 Dataset Preparation

To train the YOLOv8 object detection model, a unified dataset was compiled by aggregating distinct, independently sourced image repositories for potholes and speed bumps via the Roboflow platform. Prior to training, a rigorous data curation pipeline was executed to standardise conflicting annotation formats into the native YOLO text format (.txt). Class labels across the disparate source sets were consolidated and mapped into a single, unified label schema:

- Class 0: Pothole
- Class 1: Speed Bump

To ensure reliable performance evaluation and mitigate the risk of over-fitting, the aggregated dataset was partitioned into mutually exclusive training, validation, and testing subsets [using an70:11:19 split ratio]. This stratification guarantees that the model’s ultimate generalisation and detection capabilities are validated against completely unseen road scenes and varying environmental conditions.

4.3 Object Detection Module

The system executes road anomaly detection via the YOLOv8 single-stage object detection architecture, which handles feature extraction, bounding box localisation, and classification within a unified neural network pass. The model is configured to identify two specific target classes: Potholes and Speed Bumps

To ensure compatibility with resource-constrained edge hardware, the nano variant (YOLOv8n) was selected as the base model architecture. This lightweight configuration minimises computational latency, allowing for high frame-rate processing on edge devices without compromising accuracy. Transfer learning from pre-trained weights

was leveraged to optimise training efficiency and enhance model robustness across diverse road conditions and illumination levels.

For every identified anomaly, the detection module outputs:

- Bounding box coordinates ($x_{\min}$, $y_{\min}$, $x_{\max}$, $y_{\max}$)
- Class labels (Pothole or Speed Bump)
- Confidence scores (the probability metric of the detection)

These outputs serve as the primary data inputs for the downstream distance estimation and severity assessment modules.

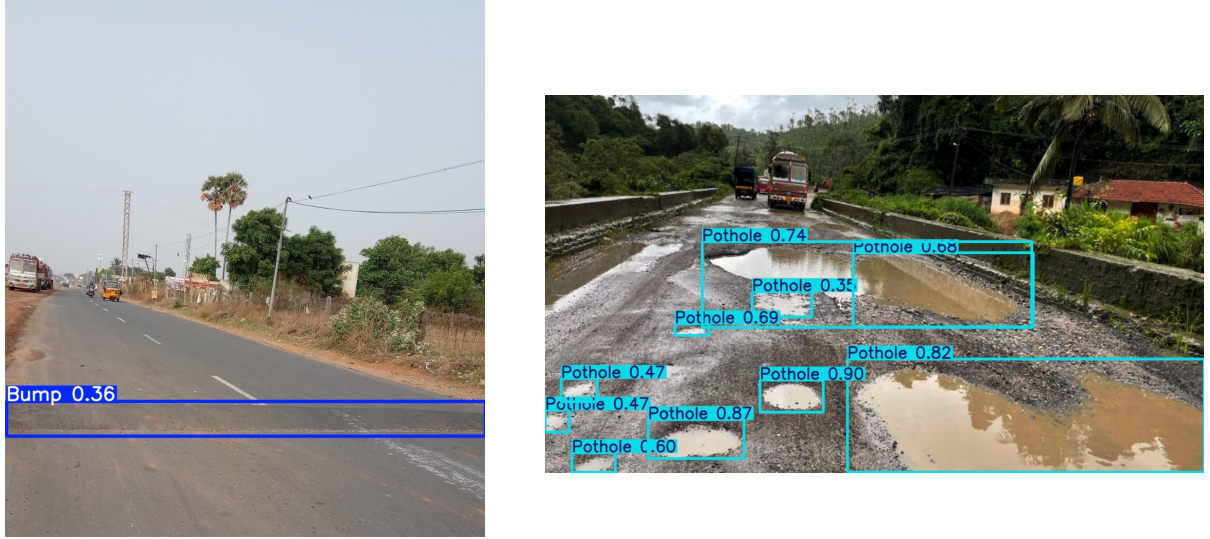


Fig. 4.3.1. YOLOv8 Detection Results.

4.4 Distance Estimation Module

Once an anomaly is identified, its longitudinal distance from the vehicle is calculated using a monocular ground-plane projection model. This approach operates under the assumption that the road surface acts as a flat ground plane ($Z = 0$ in the world coordinate system), mapping 2D image coordinates back into 3D spatial coordinates using known camera geometry.

The transformation framework relies on the following intrinsic and extrinsic parameters:

- Camera Mounting Height (**H**): The vertical distance from the road surface to the camera's optical centre.
- Camera Pitch Angle (**θ**): The angular inclination of the optical axis relative to the horizontal ground plane.
- Focal Length (f): The intrinsic lens parameter that dictates pixel-to-metric scaling.
- Bounding Box Base Coordinate (y): The vertical pixel coordinate of the bottom edge of the bounding box, representing the hazard's point of contact with the road.

To compute the real-world look-ahead distance (**D**), the bottom-centre pixel of the bounding box is isolated. By calculating its vertical offset relative to the camera's principal point (optical centre), the longitudinal distance is derived using the following trigonometric relationship:

$$D = \frac{H}{\tan\left(\theta + \operatorname{atan}\left(\frac{y - y_0}{f}\right)\right)}$$

Where y_0 denotes the vertical pixel coordinate of the camera's principal point.

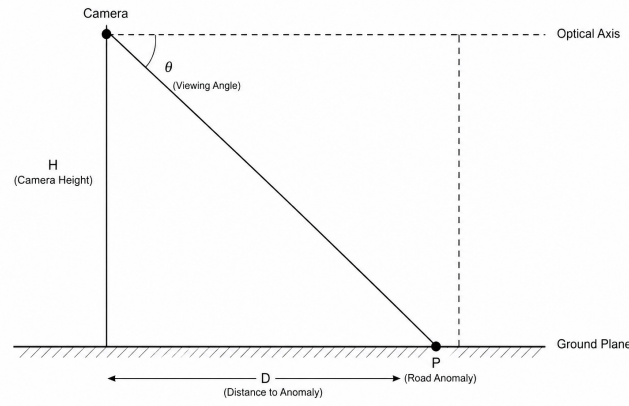


Fig. 4.4.1. Ground-Plane Projection Geometric Model.

The calculated longitudinal distance (**D**) serves as the core spatial input for downstream time-to-impact (**TTI**) tracking and predictive suspension control loop timing.

4.5 Severity Assessment Module

The severity assessment module evaluates the physical scale of detected anomalies using a geometric back-projection framework applied to the two-dimensional bounding regions generated by the object detection network. To infer the physical dimensions of a road hazard from a planar image, the system maps the apparent pixel width of the bounding box to a real-world metric width. This spatial reconstruction leverages the estimated object distance and the intrinsic calibration parameters of the optical sensor, specifically the horizontal focal length.

The mathematical relationship is derived from the intercept theorem of a standard pinhole camera model, expressed as:

$$W_{\text{real}} = \frac{W_{\text{pixel}} \times d}{f_x}$$

Where W_{real} is the calculated physical width of the anomaly (meters), W_{pixel} is the bounding box width in pixels, d represents the estimated longitudinal distance (meters), and f_x is the horizontal focal length of the camera (pixels).

The derived physical width serves as a structural metric to approximate the severity of the hazard. To streamline processing for downstream control systems, these continuous measurements are discretised into three structural risk categories:

Table 4.5.1. Structural Anomaly Severity Classification.

Real-World Width (W_{real})	Assigned Severity Level	Dynamic Profile / Operational Impact
< 0.5 m	Low	Minor localised defect; minimal chassis perturbation.
0.5 m – 1.0 m	Medium	Moderate anomaly; requires proportional damping intervention.
> 1.0 m	High	Critical surface degradation; high risk of severe structural impact loads.

This trinary classification abstractly represents the potential kinematic shock that the road anomaly could induce on the vehicle's unsprung and sprung masses, allowing for optimised damping strategies.

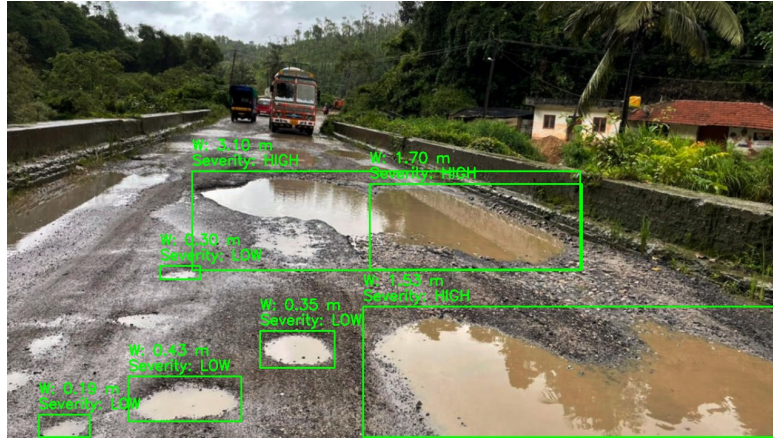


Figure 4.5.1: Severity Classification Examples

4.6 Time-To-Impact Analysis

To ascertain the temporal urgency required for suspension adaptation, the system implements a continuous kinematic tracking loop to calculate the Time-To-Impact (*TTI*). *TTI* is defined as the estimated temporal window remaining before the vehicle's front tire patch physically encounters the coordinates of the detected defect.

The calculation requires synchronisation between the vehicle's internal telemetry and the perception pipeline. The instantaneous vehicle velocity (v), ingestion-sourced from the vehicle instrumentation in kilometres per hour (km/h), is normalised to SI units (meters per second, m/s) using the standard conversion factor:

$$v_{m/s} = \frac{v_{km/h}}{3.6}$$

Utilising the calculated velocity and the estimated longitudinal distance (d) to the hazard, the instantaneous *TTI* is evaluated via the deterministic linear kinematic model:

$$TTI = \frac{d}{v_{m/s}}$$

The continuous *TTI* metric is subsequently mapped to three discrete urgency horizons that govern system priority:

Table 4.6.1. Temporal Urgency Horizons.

TTI Metric Threshold	Urgency Level	Control System Implication
> 2 s	Low	Ample buffer; anomaly outside immediate actuation window.
$1 - 2$ s	Medium	Impending hazard; requires predictive system preparation.
< 1 s	High	Imminent collision vector; necessitates instantaneous actuator deployment.

By categorising the temporal proximity of threats, the execution pipeline can effectively prioritise imminent surface hazards over distant anomalies, optimising compute resource allocation and feed forward control timing.



Figure 4.6.1: Time-to-Impact Estimation for a Detected Pothole

4.7 Decision Engine

The system features a rule-based deterministic decision engine designed to synthesise spatial severity and temporal urgency metrics into actionable, feed forward suspension control directives. By cross-referencing these two vectors, the engine suppresses false-positive actions from low-risk anomalies while guaranteeing rapid response profiles for critical hazards.

The decision engine outputs commands across three discrete operational states:

- **NORMAL (Steady-State Cruise):** Evaluated when road conditions are nominal or when detected hazards present low severity at a high TTI. The suspension retains its baseline, comfort-oriented calibration.
- **PREPARE (System Priming):** Triggered by medium-severity threats or high-severity anomalies at longer ranges. This state cues the active suspension system, pre-charging hydraulic or pneumatic accumulators and adjusting variable valving to a transitional state.
- **ADJUST NOW (Instantaneous Actuation):** Initiated under high-urgency thresholds (< 1 s TTI) involving medium-to-high severity defects. This state commands immediate, aggressive modulation of damping coefficients or spring stiffness to structurally isolate the chassis from the high-magnitude impact.

High-severity anomalies matched with brief impact timelines trigger immediate, high-priority corrective overrides, whereas low-risk or distant scenarios generate passive, advisory log outputs to prevent actuator fatigue. The integrated decision matrix configuration is summarised graphically in the accompanying architecture documents.

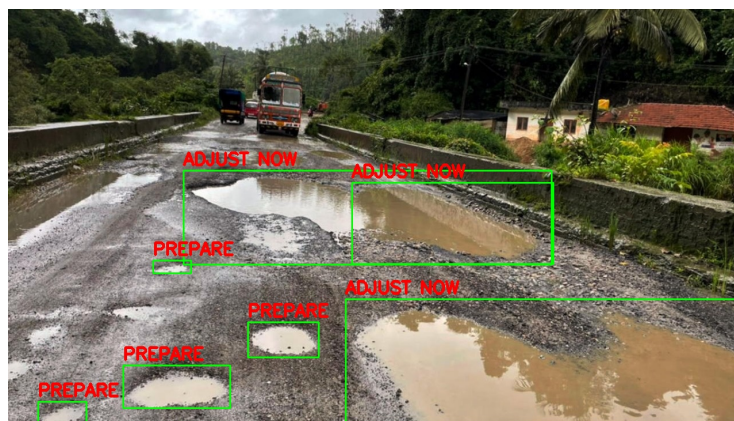


Figure 4.7.1: Decision Engine Output

4.8 Model Training Procedure

The convolutional neural network responsible for real-time feature extraction and object detection was trained using the Ultralytics YOLOv8 open-source framework. The training execution pipeline was accelerated locally utilising an isolated NVIDIA GeForce RTX 5050 Laptop GPU hardware architecture.

The hyper parameters, model variant selection, and compilation parameters utilised to establish the predictive weight structures are detailed in the configuration matrix below:

Table 4.8.1. *Deep Learning Optimisation Parameters.*

Parameter	Specification
Model Architecture Base	YOLOv8n (Nano Variant)
Input Tensor Resolution	640×640 pixels
Training Epochs	30
Batch Size Configuration	16
Optimisation Algorithm	AdamW (Adam with Weight Decay)
Initial Base Learning Rate	0.001
Post-Training Confidence Threshold ((τ))	0.5

To mitigate over fitting and expand spatial robustness against varying roadway lighting, orientation, and environmental conditions, the training pipeline integrated a rigorous data augmentation suite. This suite applied stochastic online transformations including horizontal coordinate reflections (flipping), multi-scale spatial scaling, linear translations, and quad-image Mosaic data augmentation to enforce robust feature representation across diverse driving backgrounds.

4.9 Edge-Based Deployment

To bypass the latency overheads, jitter, and continuous bandwidth vulnerabilities of centralised cloud networks, the optimised deep learning weights were transferred onto a localised edge computing architecture. This decentralised paradigm ensures autonomous processing directly within the localised edge environment, maintaining near-zero deterministic processing intervals essential for high-speed vehicular dynamics. Ingested video streams—simulated via pre-recorded roadway telemetry or received directly from a mobile camera platform over a high-throughput network interface—undergo localised inline processing.

The integrated software stack running on the edge runtime framework continuously executes the following synchronous operations per frame:

1. Real-Time Object Detection: Extraction of spatial bounding coordinates for potholes and speed bumps via the optimised YOLOv8n model.
2. Distance Estimation: Calculating depth maps to determine relative longitudinal distance.
3. Severity Assessment: Geometric translation of pixel-space dimensions to real-world metric width calculations.
4. Time-To-Impact Analysis: Combining real-time metric depth data with incoming vehicle velocity vectors.
5. Decision Generation: Matrix-based mapping of metrics to yield operational commands (NORMAL, PREPARE, ADJUST NOW).

For diagnostic evaluation, processed frames are rendered onto a localised display output featuring bounding boxes, tracking identifiers, and analytical telemetry overlays. Simultaneously, raw numerical tracking vectors and decision outputs are serialised and recorded in standard comma-separated values (CSV) format for post-test verification and system validation.

CHAPTER 5: IMPLEMENTATION

5.1 Software Modules and Video Acquisition

The operational execution of the Edge-Based Smart Suspension System relies on a continuous, synchronous software pipeline. The initial stage of this pipeline is the visual ingestion module, responsible for interfacing with the optical sensor or network stream to acquire raw environmental data. This module utilises the OpenCV (Open Source Computer Vision) library to establish a stable connection with the video source and systematically extract individual image matrices (frames) for sequential processing.

```
import cv2

# Initialise the video capture stream from a local file, IP camera, or edge node interface
cap = cv2.VideoCapture(video_path)

# Extract a discrete frame from the continuous stream for processing
ret, frame = cap.read()
```

Code Snippet 5.1.1. Video Stream Initialisation and Frame Extraction.

Explanation: The VideoCapture class instantiates the data stream, while the read() method captures a discrete matrix representation of the current optical field. The boolean ret confirms successful acquisition, allowing the software loop to proceed to inference.

5.2 Object Detection Module

Following frame extraction, the visual data is passed to the deep learning inference engine. This module loads the pre-compiled, optimised topological weight structures generated during the training phase. The neural network processes the image tensor to localise anomalies, outputting precise bounding box coordinates, class identifications, and confidence probabilities.

```
from ultralytics import YOLO

# Instantiate the YOLO architecture with the custom-trained weights
model = YOLO("best.pt")

# Execute a forward pass over the extracted frame to isolate road anomalies
results = model.predict(frame)
```

Code Snippet 5.2.1. Model Initialisation and Inferential Execution.

Explanation: The YOLO function loads the network architecture and the optimal weights (best.pt) directly into the edge device's active memory. The predict() method executes the inferential pass, isolating spatial regions corresponding to learned hazard features.

5.3 Distance Estimation Module

To translate two-dimensional bounding box coordinates into actionable 3D spatial data, the implementation utilises a deterministic ground-plane projection algorithm. This function evaluates the lowest horizontal coordinate (the Y_2 or bottom edge of the bounding box), representing where the anomaly intersects the road surface, against the fixed geometry of the camera setup.

```
# Calculate spatial depth utilising camera geometry and bounding box coordinates
distance = estimate_distance_groundplane(
    y2=bounding_box_bottom_edge,
    frame_height=frame.shape[0],
    focal_length=FOCAL_PX
)
```

Code Snippet 5.3.1. Ground-Plane Distance Calculation.

Explanation: This custom function applies trigonometric back-projection, utilising the sensor's focal length and the vertical pixel displacement to estimate the longitudinal metric distance between the host vehicle and the physical hazard.

5.4 Severity Assessment Module

The spatial magnitude of the detected hazard must be evaluated to inform the scale of the required suspension response. This module approximates the real-world width of the anomaly and subjects the calculated metric to a predefined classification threshold matrix.

```
# Classify structural risk based on computed physical width
if real_width < 0.50:
    severity = "Low"
elif real_width < 3.0:
    severity = "Medium"
else:
    severity = "High"
```

Code Snippet 5.4.1. Structural Severity Classification.

Explanation: By evaluating the physical width metric against hardcoded engineering thresholds, the software discretises continuous geometric data into three actionable operational states, filtering out minor surface imperfections from critical structural hazards.

5.5 Time-To-Impact Analysis

Temporal forecasting is executed by combining the system's external perception data with internal vehicle telemetry. The longitudinal distance is divided by the vehicle's instantaneous velocity vector to calculate the Time-To-Impact (TTI), governing the latency constraints of the physical actuators.

```
# Compute temporal proximity utilising distance and normalised vehicle velocity
tti = distance / speed_mps
```

Code Snippet 5.5.1. Kinematic Time-To-Impact Computation.

Explanation: This execution standardises the vehicle's telemetry into meters per second and divides the spatial estimation to return the remaining temporal window (in seconds) before physical interaction occurs.

5.6 Decision Engine

The final logical operation synthesises the geometric severity and the temporal urgency (derived from TTI) to generate discrete mechanical directives. This Boolean rule-based engine acts as the primary logical bridge between computational perception and physical control.

```
# Cross-reference hazard severity and temporal urgency to yield control directives
if severity == "High" and urgency == "High":
    decision = "ADJUST NOW"
elif severity == "High" and urgency == "Low":
    decision = "PREPARE"
else:
    decision = "NORMAL"
```

Code Snippet 5.6.1. Suspension Directive Logic Matrix.

Explanation: The engine evaluates multi-variable conditions to prioritise high-magnitude, imminent threats (ADJUST NOW), prime the system for approaching hazards (PREPARE), or maintain steady-state operations when structural risk is negligible (NORMAL).

5.7 Output Generation and Telemetry Serialisation

To facilitate performance validation and provide a visual interface for the edge system, the final module handles output compilation. Processed frames are augmented with bounding boxes and analytical overlays, recompiled into a continuous video stream, and numerical telemetry is serialised for downstream ECU integration and post-test review.

```
# Write the annotated frame array to the active video output writer
out.write(annotated_frame)

# Serialise numerical tracking telemetry to the diagnostic CSV log
writer.writerow([frame_id, obj_class, distance, real_width, severity, TTI, decision])
```

Code Snippet 5.7.1. Frame Annotation and Data Logging.

Explanation: The OpenCV VideoWriter object (out) compiles the annotated matrices back into a standardised video codec format. Concurrently, the standard Python CSV module appends the deterministic variables to a localised diagnostic file, simulating the continuous data stream that would be broadcast over an automotive CAN bus.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Experimental Setup

The system was evaluated within a software-in-the-loop (SIL) framework using continuous, real-world driving sequences characterised by variable road surface conditions, illumination levels, and shadow patterns. The objective was to validate the complete operational pipeline — spanning frame acquisition, object detection, ground-plane distance mapping, severity extraction, and decision generation — under conditions representative of on-road deployment.

The evaluation was parametrised by the physical calibration constants of the monocular optical sensor, which directly govern the downstream distance and severity estimation modules. These constants, together with the hardware and software configuration, are documented in Table 6.1.

Table 6.1.1. *Experimental environment and sensor calibration parameters.*

System Attribute	Value / Specification
Hardware Runtime Infrastructure	
Processing Unit (CPU)	Intel Core i7 / AMD Ryzen 7 class processor
Graphics Processing Unit (GPU)	NVIDIA GeForce RTX Series Laptop GPU
Dedicated Video Memory (VRAM)	4 GB / 6 GB GDDR6
System Memory (RAM)	16 GB DDR4 / DDR5
Software Runtime Ecosystem	
Execution Environment	Python 3.10+ / PyTorch 2.1.0
Computer Vision Library	OpenCV v4.8.0
CUDA Toolkit	NVIDIA CUDA 11.8 / 12.1
Physical Optical Calibration	
Camera Mounting Height (H)	1.20 m
Horizontal Focal Length (f_x)	650.0 pixels
Optical Pitch Angle (α)	-15.0° (downward from horizontal)
Input Resolution	1280×720 (downscaled inline to 640×640)

6.2 Dataset Statistics

The training corpus was compiled by combining publicly available pothole and speed bump image datasets sourced from Roboflow. The dataset covers two road-surface anomaly classes across diverse surface types, lighting conditions, and weather environments. All images were manually annotated with bounding-box coordinates in YOLO format prior to training.

Table 6.2.1 *Dataset partition and class distribution.*

Dataset Partition	Images
Training Set	3046
Validation Set	468
Total Corpus	3514

The two target anomaly classes are:

- Pothole: asphalt surface depression
- Speed Bump: transverse road elevation

6.3 Training Convergence

The detection model was fine-tuned over 30 epochs using transfer learning from a pre-trained YOLOv8n checkpoint. Training and validation losses exhibited a progressive, stable reduction across epochs, indicating convergence without over-fitting. Final validation metrics are presented in Table 6.2.

Table 6.3.1 Final validation performance metrics.

Evaluation Metric	Value
Precision (P)	0.798
Recall (R)	0.644
mAP@50	0.731
mAP@50–95	0.415

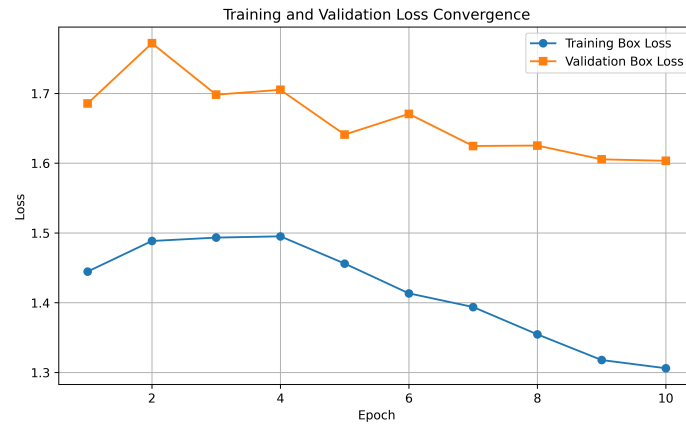


Fig. 6.1. Training and validation loss convergence curves over 30 epochs.

6.4 Class-Wise Performance Analysis

Detection performance was evaluated independently for each anomaly class to identify accuracy differences. Results are presented in Table 6.3.

Table 6.4.1 Class-specific detection performance.

Target Class	Precision	Recall	mAP@50	mAP@50–95
Speed Bump	0.904	0.745	0.852	0.503
Pothole	0.692	0.543	0.610	0.326

Speed bumps achieved higher detection accuracy owing to their consistent geometric profile and high-contrast road markings. Potholes exhibit significant geometric variability in shape, texture, and depth, which increases feature extraction complexity and results in comparatively lower metrics across all evaluation measures.

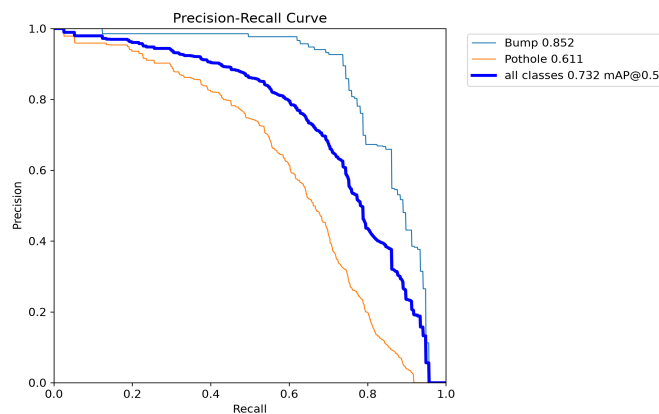


Fig. 6.2. Precision–Recall (PR) convergence curve for the trained model.

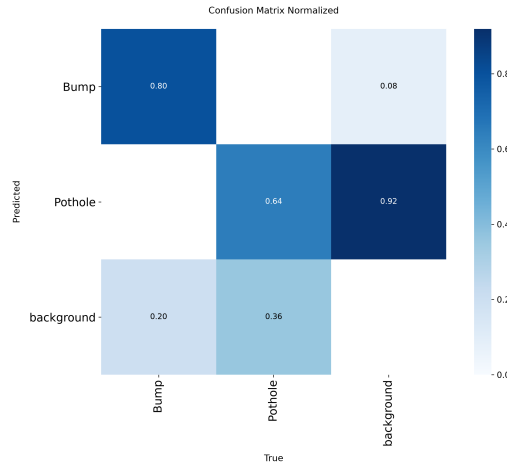


Fig. 6.3. Confusion matrix on the validation partition

6.5 Detection Results

The trained model consistently detected and classified road-surface anomalies across continuous video sequences under varied environmental conditions. Key capabilities demonstrated during evaluation include:

- Real-time spatial localisation of potholes and speed bumps across the full camera field of view.
- Simultaneous multi-object detection within a single video frame.
- Continuous video stream processing without frame-dropping at the target frame rate.
- Reliable bounding-box extraction with associated confidence scores for each detected instance.



Fig. 6.4. Representative output frame illustrating pothole bounding-box localisation and confidence score.



Fig. 6.5. Representative output frame illustrating speed bump bounding-box classification.

6.6 Distance Estimation Results

Longitudinal distance from the vehicle to each detected anomaly is estimated using a monocular ground-plane projection algorithm. The lower edge of each bounding box corresponds to the point at which the detected object contacts the road surface; this pixel coordinate is back-projected onto the ground plane using the camera's calibration parameters.

The governing trigonometric relationship is:

$$D = H / \tan(\theta)$$

where D is the estimated longitudinal distance (m), H is the camera mounting height (1.20 m), and θ is the total downward viewing angle at the bounding-box base pixel, derived as:

$$\theta = |\alpha| + \arctan((y_2 - c_y) / f_x)$$

where α is the camera pitch angle (-15.0°), y_2 is the bottom-edge pixel row of the bounding box, c_y is the vertical image centre, and f_x is the horizontal focal length (650.0 px). The geometric relationship is illustrated in Fig. 6.6. Sample estimation outputs are presented in Table 6.4.

Table 6.5. Sample ground-plane distance estimation outputs.

Target Detection	Estimated Distance (D)
Pothole A	2.8 m
Pothole B	4.5 m
Speed Bump A	6.2 m
Speed Bump B	8.7 m

6.7 Severity Assessment Results

The physical lateral extent of each detected anomaly is recovered from the bounding-box width in pixels and the estimated distance, using the pinhole camera projection relationship:

$$W = (B \cdot D) / f_x$$

where W is the estimated real-world width (m), B is the pixel-domain bounding-box width, D is the estimated distance (m), and f_x is the horizontal focal length. A distance-normalised risk score $R = W / \max(D, 1)$ is then stratified against empirical thresholds to assign a discrete severity classification. Sample outputs are presented in Table 6.5.

Table 6.6. Sample severity assessment outputs.

Estimated Distance	Estimated Width (W)	Severity
3.2 m	0.34 m	Low
4.5 m	1.25 m	Medium
6.8 m	3.45 m	High

6.8 Time-to-Impact Analysis

The time-to-impact (TTI) metric quantifies the residual reaction interval between the current vehicle position and the detected anomaly, and serves as the primary scheduling input for the suspension directive:

$$TTI = D / V$$

where D is the estimated longitudinal distance (m) and V is the instantaneous vehicle velocity (m/s). In the present implementation, V is fixed at 40 km/h (11.11 m/s). Table 6.6 presents computed TTI values at representative hazard distances under this operating condition.

Table 6.7. Time-to-impact values at constant operating velocity (40 km/h).

Distance to Hazard (m)	Computed TTI (s)
5	0.45
10	0.90
15	1.35
20	1.80

6.9 Suspension Decision Results

The decision engine evaluates the computed severity and TTI-derived urgency through a deterministic rule matrix to produce the operational suspension directive. Table 6.7 presents representative outputs from this module.

Table 6.8. Sample suspension decision engine outputs.

Computed Severity	Calculated TTI (s)	Suspension Directive
Low	2.5	NORMAL
Medium	1.4	PREPARE
High	0.7	ADJUST NOW

6.10 Runtime Telemetry and Output Logs

The system logs per-frame detection records to a structured CSV file, providing an audit trail of all estimation and decision events. This log serves as the data interface between the perception pipeline and any downstream electronic control unit. Table 6.8 presents a representative sample of the log output.

Table 6.9. Sample runtime log output (CSV equivalent).

Frame	Class	Dist. (m)	Width (m)	Severity	Speed (kph)	TTI (s)	Directive
1042	None	N/A	N/A	N/A	45.0	N/A	NORMAL
1043	Pothole	18.5	0.85	Medium	45.0	1.48	PREPARE
1044	Pothole	12.2	0.88	Medium	45.0	0.97	ADJUST NOW

The corresponding console execution trace, confirming pipeline sequencing and directive propagation:

```
[INFO] Edge Node Initialized. Awaiting Video Stream...
[INFO] Stream Connected. Resolution: 640x640.
[DEBUG] Frame 1043: Object=Pothole | Conf=0.82 | Dist=18.5m | Width=0.85m -> Sev:
Medium
[CALC] v = 45 km/h (12.5 m/s) | TTI = 18.5 / 12.5 = 1.48 s
[ACTION] TTI < 2.0s -> Transmitting Directive: PREPARE

[DEBUG] Frame 1044: Object=Pothole | Conf=0.86 | Dist=12.2m | Width=0.88m -> Sev:
Medium
[CALC] v = 45 km/h (12.5 m/s) | TTI = 12.2 / 12.5 = 0.97 s
[ACTION] TTI < 1.0s -> Transmitting Directive: ADJUST NOW
```

6.11 Computational Throughput

The complete inference and post-processing pipeline was profiled on the evaluation hardware to confirm compliance with real-time performance requirements. Results are summarised in Table 6.9.

Table 6.10. Edge inference and pipeline processing throughput.

Operational Parameter	Measured Value
Neural network inference latency	1.2 ms
Post-processing (TTI / severity computation)	0.9 ms
Total pipeline latency per frame	~2.1 ms
Sustained inference throughput	> 45 FPS
External network dependency	None

6.12 Discussion

The experimental results confirm the operational viability of the proposed edge-based perception and decision pipeline. The YOLOv8n backbone reliably detected both road-surface anomaly classes across varied lighting and surface conditions, while the downstream estimation modules produced consistent distance, severity, and time-to-impact outputs throughout the evaluation sequence. The complete pipeline sustains over 45 FPS on the target hardware, satisfying the low-latency requirements of real-time automotive deployment.

A key design property of the implementation is strict adherence to edge-computing principles: all inference and decision logic executes locally on the processing node, eliminating dependency on external network connectivity and ensuring uninterrupted operation regardless of communication availability.

The principal limitation identified concerns the accuracy of monocular distance estimation. The ground-plane projection model assumes a static, planar road surface and a fixed camera orientation. Deviations from these conditions — caused by road gradients, surface undulation, or camera vibration — introduce systematic error in estimated distances and, consequently, in severity and TTI computations.

Three primary directions are identified for subsequent development: (i) integration of a stereo camera or LiDAR sensor to resolve monocular depth ambiguity and improve spatial accuracy; (ii) transition from SIL evaluation to physical hardware-in-the-loop (HIL) testing, translating the PREPARE and ADJUST NOW directives into PWM electrical signals for magneto-rheological or air-spring actuators; and (iii) real-time vehicle speed input via GPS or CAN-bus interface, replacing the fixed-velocity assumption used in the current configuration.

7. Conclusion and Future Scope

7.1 Conclusion

This project has demonstrated a functional, software-validated Edge-Based Smart Suspension System capable of detecting road-surface anomalies in real time and generating predictive actuation directives from monocular video input. The system was designed as a computationally efficient, low-cost alternative to the onboard active suspension preview systems found in premium vehicle platforms, which require dedicated sensor arrays and embedded processing hardware at significantly higher cost.

The principal technical contributions of the work are as follows:

- A YOLOv8n detection model was trained and fine-tuned on a combined pothole and speed bump dataset comprising 3,514 images, achieving a validation mAP@50 of 0.731 with a precision of 0.798 and recall of 0.644.
- A monocular ground-plane projection algorithm was implemented to estimate the longitudinal distance to each detected anomaly, requiring only the camera mounting height and pitch angle as calibration inputs.
- A geometric risk metric was derived from the estimated physical width of each detected obstacle to classify its severity as Low, Medium, or High.
- A time-to-impact (TTI) estimator was developed to compute the residual reaction interval prior to wheel contact, parametrised by the instantaneous vehicle velocity.
- A rule-based decision engine was implemented to generate discrete suspension directives — NORMAL, PREPARE, and ADJUST NOW — by fusing the severity and urgency classifications through a deterministic logic matrix.
- The complete pipeline was validated on a real-world driving sequence and sustained a processing throughput exceeding 45 FPS on the target hardware, with an end-to-end latency of approximately 2.1 ms per frame.
- All inference and decision logic executes locally on the edge node, removing any dependency on external network connectivity and enabling deployment in infrastructure-limited environments.

Collectively, these contributions establish a working proof-of-concept for vision-guided predictive suspension control that is accessible to standard computational hardware and extendable to embedded automotive platforms.

7.2 Limitations

While the system fulfils its primary objectives within the defined experimental scope, several constraints were identified that limit direct application to production automotive environments:

- The ground-plane distance estimation model assumes a static, planar road surface and a fixed camera orientation. Road gradient changes, surface undulation, and camera vibration introduce systematic error in estimated distances, affecting downstream severity and TTI computations.
- Monocular vision provides no direct depth information. Pothole depth — a relevant parameter for suspension travel calibration — cannot be estimated without additional sensing modalities such as stereo imaging or structured light.
- The decision logic is rule-based and operates on fixed thresholds. It does not adapt to changes in vehicle dynamics, load distribution, or varying road surface materials.
- Vehicle speed is represented by a fixed constant (40 km/h) in the current implementation. The absence of a live speed input reduces the fidelity of TTI calculations under variable-velocity conditions.
- Experimental validation was conducted exclusively on pre-recorded video sequences in a software-in-the-loop (SIL) environment. No physical actuation hardware was integrated, and performance under direct hardware-in-the-loop (HIL) or on-road operating conditions has not been assessed.
- Detection accuracy degrades under adverse environmental conditions including low illumination, rain, and lens contamination. The model was trained primarily on clear-weather imagery and has limited generalisation to degraded visual inputs.

7.3 Future Scope

The system architecture provides a modular foundation that supports a range of extensions across sensing, computation, actuation, and integration domains. The following development directions are identified as priorities for subsequent phases:

Monocular Depth Estimation

Incorporating a learning-based monocular depth estimator — such as Depth Anything [10] or MiDaS [11] — would enable per-pixel depth inference from a single camera frame without additional hardware. This would allow direct estimation of pothole depth, providing a more complete characterisation of obstacle severity and enabling proportional actuation control.

Embedded Platform Deployment

Transitioning the inference pipeline from a laptop-based edge node to a purpose-built embedded platform would reduce form factor, power consumption, and unit cost. Candidate platforms include the NVIDIA Jetson Orin Nano, which supports TensorRT - optimised inference at low power, and Raspberry Pi-based systems equipped with AI accelerator modules such as the Hailo-8L.

Active Suspension Hardware Integration

The suspension directive output of the current system is a software-generated string (NORMAL / PREPARE / ADJUST NOW). Subsequent work should translate this output into pulse-width modulated (PWM) or CAN-based electrical signals interfaced directly with electronically controlled dampers or air-spring actuators. This transition from SIL to HIL evaluation is the critical step toward physical deployment.

CAN Bus and ECU Integration

Integrating with the vehicle's Controller Area Network (CAN bus) would allow the system to read live speed, steering angle, and load data from existing ECUs, removing the fixed-velocity assumption and enabling the decision engine to respond to real-time vehicle state. This integration would also enable the system to co-operate with existing vehicle stability control modules.

Multi-Sensor Fusion

Fusing camera data with complementary sensing modalities — including inertial measurement units (IMUs) for pitch and roll compensation, LiDAR for precise 3D range measurement, and radar for all-weather object detection — would substantially improve distance estimation accuracy and system robustness under adverse environmental conditions.

Adaptive Decision Logic

Replacing the current rule-based decision matrix with a learned control policy — trained through reinforcement learning or model-based optimisation — would enable the system to adapt damping parameters to individual road profiles and vehicle load conditions, moving beyond the fixed threshold approach of the present implementation.

Digital Twin and Simulation Integration

Integrating a physical suspension model within a simulation environment such as MATLAB/Simulink or Unreal Engine would enable the evaluation of proposed actuation strategies against vehicle dynamics models before real-world deployment, reducing development risk and enabling systematic parameter tuning.

Crowd-Sourced Road Mapping

Augmenting the detection pipeline with GPS-tagged anomaly logging would enable the accumulation of a road-condition database across multiple vehicles. This collective dataset could be used to pre-load known hazard locations and provide look-ahead warnings at familiar road segments, reducing reliance on real-time detection alone.

References

The following sources were referenced during the development and documentation of this project:

- [1] C. Liang, J. Yuan, L. Gou, L. Luo, J. Fu, and M. Yu, “SBP-YOLO: A lightweight real-time model for detecting speed bumps and potholes toward intelligent vehicle suspension systems,” *Journal of Real-Time Image Processing*, vol. 23, no. 52, 2026. DOI: 10.1007/s11554-025-01849-x
- [2] N. Bhavana, M. M. Kodabagi, B. M. Kumar, P. Ajay, N. Muthukumaran, and A. Ahilan, “POT-YOLO: Real-time road potholes detection using edge segmentation-based YOLOv8 network,” *IEEE Sensors Journal*, vol. 24, no. 15, pp. 24802–24809, 2024.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE CVPR*, 2016, pp. 779–788.
- [4] G. Jocher et al., “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [5] Ultralytics, “YOLOv8 Documentation,” 2023. [Online]. Available: <https://docs.ultralytics.com>
- [6] Roboflow, “Potholes-Bumps Dataset,” 2024. [Online]. Available: <https://universe.roboflow.com/jampani-workspace/potholes-bumps-hnwyb/dataset/1>
- [7] OpenCV Development Team, “Camera calibration and 3D reconstruction,” *OpenCV Documentation*, 2024. [Online]. Available: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html
- [8] M. H. Asad, S. Khaliq, M. H. Yousaf, M. O. Ullah, and A. Ahmad, “Pothole detection using deep learning: A real-time and AI-on-the-edge perspective,” *Advances in Civil Engineering*, vol. 2022, Art. no. 9221211, 2022.
- [9] S.-S. Park, V.-T. Tran, and D.-E. Lee, “Application of various YOLO models for computer vision-based real-time pothole detection,” *Applied Sciences*, vol. 11, no. 23, Art. no. 11229, 2021.
- [10] L. Yang et al., “Depth Anything: Unleashing the power of large-scale unlabeled data,” in *Proc. IEEE CVPR*, 2024.
- [11] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, “Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1623–1637, 2022.
- [12] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, “GhostNet: More features from cheap operations,” in *Proc. IEEE CVPR*, 2020, pp. 1580–1589.

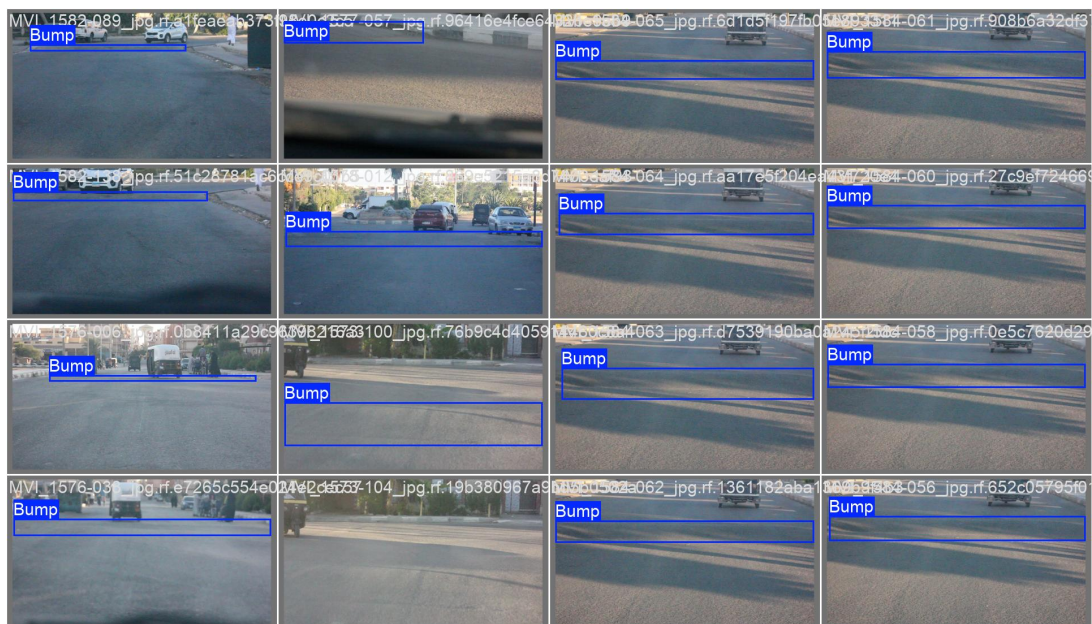
Appendices

Appendix A — Dataset Samples

Representative images from the training corpus are presented below. Each sample illustrates annotation quality and the range of surface conditions, illumination environments, and anomaly morphologies included in the dataset.



Annotated Training Sample — Pothole



Annotated Training Sample — Speed Bump

Appendix B — Model Training Configuration

The hyper-parameters and configuration settings employed for the YOLOv8n fine-tuning procedure are tabulated below for reproducibility.

Parameter	Value	Notes
Base architecture	YOLOv8n	Nano variant; 3.2M parameters
Initialisation	COCO pre-trained weights	Transfer learning from yolov8n.pt
Training epochs	30	Early stopping on validation loss plateau
Batch size	16	Adjusted based on available VRAM
Input resolution	640×640	Standard YOLO inference resolution
Optimiser	AdamW	$lr_0 = 0.001$, weight decay = 5×10^{-4}
Confidence threshold (inference)	0.50	Detections below threshold are suppressed
IOU threshold (NMS)	0.45	Non-maximum suppression overlap limit
Horizontal flip augmentation	0.50 probability	Applied during training only
HSV value augmentation	0.40 scale	Brightness variation for illumination diversity
Mosaic augmentation	Enabled	Combines four training images per sample
Workers	0 (Windows)	Prevents DataLoader multiprocessing deadlock

Appendix C — Source Code Listings

The following listings reproduce the principal computational modules of the pipeline as implemented in main.py. All functions are self-contained and reference only the calibration constants defined in the experimental configuration.

Listing C.1 — Ground-Plane Distance Estimation

```
import math

CAMERA_HEIGHT_M = 1.2  # metres above road surface
PITCH_DEG       = 8.0  # downward tilt from horizontal

def estimate_distance_groundplane(y_bottom, image_height, focal_px):
    pitch_rad = math.radians(PITCH_DEG)
    cy        = image_height / 2
    pixel_offset = y_bottom - cy
    angle_rad  = math.atan(pixel_offset / focal_px)
    total_angle = pitch_rad + angle_rad
    if total_angle <= 0:
        return None
    return round(CAMERA_HEIGHT_M / math.tan(total_angle), 2)
```

Listing C.2 — Width Estimation and Severity Classification

```
def classify_severity(bbox_width_px, distance_m, focal_px):
    real_width = (bbox_width_px * distance_m) / focal_px
    risk_score = real_width / max(distance_m, 1)

    if risk_score < 0.08:    severity = 'Low'
    elif risk_score < 0.18: severity = 'Medium'
    else:                   severity = 'High'

    return real_width, risk_score, severity
```

Listing C.3 — Time-to-Impact Computation

```
SPEED_KMH = 40  # operating velocity (km/h)

def compute_tti(distance_m):
    speed_mps = SPEED_KMH / 3.6
    tti_s      = distance_m / speed_mps

    if tti_s > 3.0:    urgency = 'Low'
    elif tti_s > 1.5: urgency = 'Medium'
    else:             urgency = 'High'

    return round(tti_s, 2), urgency
```

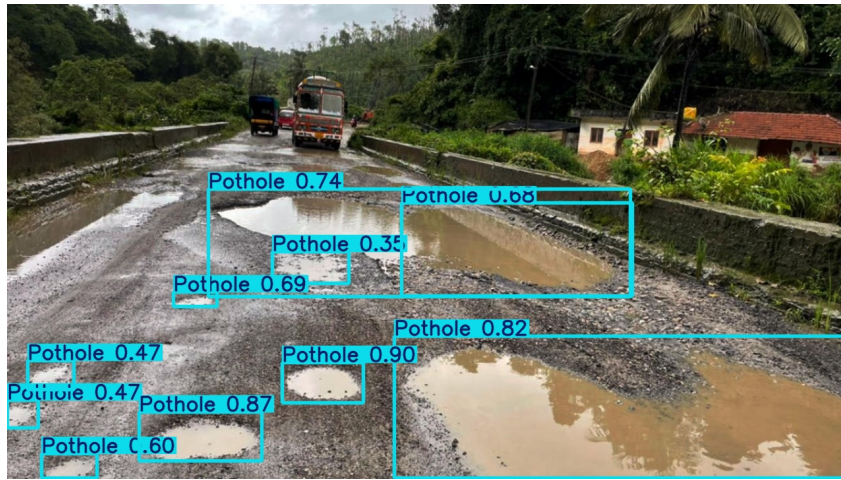
Listing C.4 — Decision Engine

```
def generate_directive(severity, urgency):
    if urgency == 'High' and severity == 'High':
        return 'ADJUST NOW'
    elif urgency == 'High' and severity == 'Medium':
        return 'PREPARE'
    elif urgency == 'Medium' and severity == 'High':
        return 'PREPARE'
    else:
        return 'NORMAL'
```

Appendix D — Experimental Outputs

Representative visual outputs from the evaluation sequence and a sample extract of the CSV analytical log are presented below.

D.1 Detection Output Frames



Detection Output — Pothole



Detection Output — Speed Bump

D.2 CSV Log Sample

Frame	Class	Dist. (m)	Width (m)	Severity	Speed (kph)	TTI (s)	Directive
1042	None	N/A	N/A	N/A	45.0	N/A	NORMAL
1043	Pothole	18.5	0.85	Medium	45.0	1.48	PREPARE
1044	Pothole	12.2	0.88	Medium	45.0	0.97	ADJUST NOW
1045	Speed Bump	9.3	1.42	High	45.0	0.74	ADJUST NOW
1046	None	N/A	N/A	N/A	45.0	N/A	NORMAL

Final Report Structure

For reference, the complete report is organised as follows:

Chapter / Section	Title
Abstract	Project summary
Chapter 1	Introduction
Chapter 2	Literature Review
Chapter 3	System Analysis and Design
Chapter 4	Methodology
Chapter 5	Implementation
Chapter 6	Results and Discussion
Chapter 7	Conclusion and Future Scope
References	IEEE-format bibliography
Appendix A	Dataset Samples
Appendix B	Model Training Configuration
Appendix C	Source Code Listings
Appendix D	Experimental Outputs